

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства
Intel – разработка ассемблерной вставки»

Выполнила:

Студентка 4 курса

Группы АС-66

Прокурат В.Д.

Проверила:

Кулик А.Д.

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками.

Вариант 20 (4)

Задание: 1. Изучить следующие теоретические сведения: программная модель микропроцессора; регистры и их назначение; организация памяти; основные конструкции и базовые команды ассемблера (используя материалы лекций и методические указания).

2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, выданному преподавателем. Оформить ее в виде отдельной функции.

3. Реализовать данную обработку строки также в виде функции на C++. Сравнить быстродействие обоих вариантов.

Вариант задания: даны 2 строки. Совместить четные символы одной строки с нечетными другой.

Код программы (функция с ассемблерной вставкой):

```
void asmCombine(const char* s1, const char* s2, char* res) {
    __asm {
        push esi
        push edi

        mov esi, s1
        mov edi, res
        xor ecx, ecx

    loop1 :
        mov al, [esi + ecx]
        test al, al
        jz end1
        test ecx, 1
        jnz skip1
        mov[edi], al
        inc edi
        skip1 :
        inc ecx
        jmp loop1

        end1 :
        mov esi, s2
        xor ecx, ecx

        loop2 :
        mov al, [esi + ecx]
        test al, al
        jz end2
        test ecx, 1
        jz skip2
        mov[edi], al
        inc edi
        skip2 :
        inc ecx
        jmp loop2

        end2 :
        mov byte ptr[edi], 0

        pop edi
        pop esi
    }
}
```

Код программы (функция на c++):

```
void cppCombine(const char* s1, const char* s2, char* res) {
    int k = 0;
    for (int i = 0; s1[i] != '\0'; ++i) {
        if (i % 2 == 0) {
            res[k++] = s1[i];
        }
    }
    for (int i = 0; s2[i] != '\0'; ++i) {
        if (i % 2 == 1) {
            res[k++] = s2[i];
        }
    }
}
```

```

        }    res[k++] = s2[i];
    }
    }
    res[k] = '\\0';
}

```

Результат работы программы:

```

Консоль отладки Microsoft V  X  +  v

[ASM]  s1 = "pnreoukoupr"
[ASM]  s2 = "bowrk2l2m8"
[ASM]  результат: prokuror228

[C++]  s1 = "pnreoukoupr"
[C++]  s2 = "bowrk2l2m8"
[C++]  результат: prokuror228

>> Если всё верно, выше должно быть: prokuror228
>> Если нет – значит где-то потерялся байт. Ищи его, он рядом.

----- замеры скорости -----
ASM:   0.281 сек
C++:   0.728 сек
Вывод: ассемблер быстрее в 2.59075 раз.

[программа завершена, можно закрывать]

D:\politech\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab1\lab1\Debug\lab1.exe (
Нажмите любую клавишу, чтобы закрыть это окно...

```

Вывод: в ходе работы изучена программная модель микропроцессора Intel, назначение регистров, организация памяти и базовые команды ассемблера. Была написана ассемблерная вставка и аналогичная функция на C++, обе совмещают чётные символы первой строки с нечётными символами второй. Корректность подтверждена одинаковым результатом. Сборка выполнялась в Debug (без оптимизаций, с отладочной информацией) и Release (с оптимизацией по скорости), чтобы сравнить, как компилятор влияет на быстродействие. Замеры показали, что ассемблер быстрее в обеих конфигурациях, так как в C++ проверка чётности идёт через операцию взятия остатка, а в ассемблере — одной инструкцией `test`. Ассемблер даёт точный контроль над регистрами и памятью, но проигрывает C++ в читаемости и переносимости, поэтому нужен только для критичных по скорости участков.